

Parallelization of Singular Value Decomposition

Aaryan
co21btech11001

Aayush Kumar
co21btech11002

Harshit Shambharkar
me21btech11019

Loukik Kalbande
me21btech11028

Raman Kumar
me21btech11043

1 Introduction

The Singular Value Decomposition (SVD) is a fundamental matrix factorization technique widely used in numerical linear algebra, with applications ranging from signal processing and machine learning to computational physics. As matrix sizes grow in modern scientific and engineering computations, the computational cost of SVD becomes a significant bottleneck. Parallelizing the SVD computation distributes the workload across multiple processors, enabling scalability for large-scale problems. This work explores strategies for parallelizing key components of the SVD computation such as matrix multiplication, QR decomposition, and bidiagonalization - aiming to reduce execution time while maintaining numerical stability and accuracy. By carefully analyzing for computational load distribution, we develop a parallel implementation suited for high-performance computing environments.

Let $A \in \mathbb{R}^{m \times n}$ be a real matrix. The **Singular Value Decomposition** of A is given by:

$$A = U\Sigma V^T$$

where:

- $U \in \mathbb{R}^{m \times m}$ is an orthogonal matrix ($U^T U = I$)
- $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix with non-negative real numbers on the diagonal
- $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix ($V^T V = I$)

The diagonal entries $\sigma_i = \Sigma_{ii}$ are called the **singular values** of A , and they satisfy:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$$

2 Algorithms

To derive this using eigendecomposition:

Step 1: Form $A^T A$ and AA^T

$$A^T A = V \Lambda V^T \quad (\text{Eigendecomposition of } A^T A)$$

$$AA^T = U \Lambda' U^T \quad (\text{Eigendecomposition of } AA^T)$$

Here, V contains eigenvectors of $A^T A$, and Λ contains the corresponding eigenvalues $\lambda_i \geq 0$.

Step 2: Singular Values from Eigenvalues

Let the eigenvalues λ_i of $A^T A$ be sorted as:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0$$

Then the singular values are:

$$\sigma_i = \sqrt{\lambda_i}$$

Step 3: Constructing V , Σ , and U

Let $V = [v_1, v_2, \dots, v_n]$, where v_i are eigenvectors of $A^T A$.

Then the corresponding left singular vectors u_i (columns of U) are given by:

$$u_i = \frac{1}{\sigma_i} A v_i, \quad \text{for } \sigma_i \neq 0$$

Finally, construct Σ as a diagonal matrix with the singular values σ_i on the diagonal.

Final Result

$$A = U \Sigma V^T$$

2.1 QR Algorithm

The QR algorithm is a fundamental method for computing the eigenvalues of a square matrix. It proceeds by iteratively factorising the matrix A into a product $A = QR$, where Q is orthogonal and R is upper triangular, and then forming a new matrix $A' = RQ$. This process preserves similarity and gradually drives the matrix towards an upper triangular form, where the diagonal entries approximate the eigenvalues. The algorithm is widely used due to its stability and general applicability. We implement both serial and parallel versions of this algorithm.

2.1.1 Serial

In the serial version of the QR algorithm, each iteration transforms the matrix to a form closer to upper triangular, whose diagonal converges to the eigenvalues of the original matrix. The main steps are:

- The matrix A ($A = D^T D$, where D is the input matrix) is decomposed into Q (orthogonal) and R (upper triangular) using the Modified Gram-Schmidt process.
- The new matrix is computed as $A \leftarrow RQ$, preserving similarity to the original matrix.
- The process is repeated until the matrix converges to upper triangular form.
- The eigenvalues are obtained from the diagonal elements of the upper triangular matrix, and the corresponding eigenvectors are calculated.
- The matrix V is formed by stacking the eigenvectors as columns, while the columns of matrix U are computed as DV_j/σ_j , with remaining columns orthogonalized via Gram-Schmidt.
- The singular values are derived from the eigenvalues of the matrix $A = D^T D$, and the matrix Σ is constructed as a diagonal matrix containing the singular values.

The time complexity per iteration is $\mathcal{O}(\frac{m^2 n^2}{\max(m, n)})$ for an $m \times n$ matrix. The pseudo code is mentioned in Algorithm 1.

Algorithm 1 Serial SVD Algorithm using QR

```
1: procedure SVD( $D$ )
2:    $[M, N] \leftarrow \text{size}(D)$ 
3:    $A \leftarrow D^T D$  ▷ Compute covariance matrix
4:   Compute eigenvalues/vectors:
5:    $[A, V] \leftarrow \text{QR\_ITERATION}(A)$  ▷ Get eigenvalues and eigenvectors
6:    $\lambda_i \leftarrow A_{ii}$  for  $i = 1, \dots, N$  ▷ Extract eigenvalues
7:   Compute SVD components:
8:   for  $i = 1$  to  $N$  do
9:      $\sigma_i \leftarrow \sqrt{\max(\lambda_i, 0)}$ 
10:  end for
11:   $\Sigma \leftarrow \text{diag}(\sigma_1, \dots, \sigma_N)$ 
12:   $U \leftarrow \text{zeros}(M, N)$ 
13:  for  $i = 1$  to  $N$  do
14:     $U[:, i] \leftarrow DV[:, i] / \sigma_i$ 
15:  end for
16:  return  $U, \Sigma, V^T$ 
17: end procedure

18: function QR_ITERATION( $A$ ) ▷ QR iteration with MGS for eigen decomposition
19:    $V \leftarrow I_N$  ▷ Initialize eigenvector matrix
20:   for  $iter = 1$  to  $\text{max\_iterations}$  do
21:      $[Q, R] \leftarrow \text{MGS}(A_k)$  ▷ QR decomposition
22:      $A_{k+1} \leftarrow RQ$  ▷ Update A
23:      $V \leftarrow VQ$  ▷ Update eigenvectors
24:     if  $\text{norm\_diag\_diff}(A_{k+1}, A_k) < \text{tolerance}$  then
25:       break
26:     end if
27:   end for
28:   return  $A, V$  ▷ Return eigenvalues and eigenvectors
29: end function

30: function MGS( $A$ ) ▷ Modified Gram-Schmidt orthogonalization
31:    $[m, n] \leftarrow \text{size}(A)$ 
32:    $Q \leftarrow \text{zeros}(m, n), R \leftarrow \text{zeros}(n, n)$ 
33:   for  $i = 1$  to  $n$  do
34:      $v_i \leftarrow A(:, i)$ 
35:   end for
36:   for  $i = 1$  to  $n$  do
37:      $r_{ii} \leftarrow \|v_i\|_2$ 
38:      $q_i \leftarrow \frac{v_i}{r_{ii}}$ 
39:     for  $j = i + 1$  to  $n$  do
40:        $r_{ij} \leftarrow q_i^T v_j$ 
41:        $v_j \leftarrow v_j - r_{ij} q_i$ 
42:     end for
43:   end for
44:   return  $Q = [q_1, q_2, \dots, q_n], R = [r_{ij}]$ 
45: end function
```

2.1.2 Parallel

In the parallel version of the QR-based SVD algorithm, the computational load is distributed across multiple threads using OpenMP to accelerate the QR decomposition and matrix multiplications. The parallelization

strategy includes:

- The matrix $A = D^\top D$ is computed in parallel using dot products of the rows/columns of D .
- The Modified Gram-Schmidt (MGS) process is parallelized to orthogonalize columns and compute Q and R using OpenMP `parallel for` loops.
- Matrix multiplication $A \leftarrow RQ$ and update of the eigenvector matrix $V \leftarrow VQ$ are also parallelized.
- The convergence check is parallelized by computing the norm of diagonal differences in parallel.
- The final construction of U and normalization by singular values σ_j are computed using parallel for-loops.

The overall time complexity per iteration remains $\mathcal{O}(\frac{m^2 n^2}{\max(m, n)})$, but parallelization reduces the wall-clock time significantly depending on the number of threads used. The pseudo code is mentioned in Algorithm 2.

2.2 Golub-Kahan’s Algorithms

The Golub-Kahan algorithm is a well-known method for computing the Singular Value Decomposition (SVD) of a matrix. It proceeds in two main stages:

1. **Bidiagonalization:** Reducing a general matrix to bidiagonal form using Householder transformations.
2. **Diagonalization:** Applying an implicit QR iteration to diagonalize the bidiagonal matrix, thereby obtaining singular values and singular vectors.

We implement both serial and parallel versions of this algorithm. The parallelization exploits independent computations within vector operations and matrix updates using OpenMP.

2.2.1 Serial

In the serial version, each step of the algorithm is executed sequentially:

- Householder reflections are applied from the left and right to zero out below-diagonal and above-superdiagonal entries, respectively.
- A sequence of implicit QR steps with Givens rotations is used to drive off-diagonal entries to zero and extract singular values.
- Matrices U and V accumulate the transformations to form the left and right singular vectors, respectively.

The time complexity of each iteration is $\mathcal{O}(n(n + m))$.

The pseudo-code for the serial version is shown in Algorithm 3.

2.2.2 Parallel

The parallel version uses OpenMP pragmas to exploit data-level parallelism available in many of the steps:

- Vector norms and dot products in Householder reflection are parallelized.
- Matrix updates from Householder reflections and Givens rotations are parallelized across rows and columns.
- Singular value convergence checks operate in parallel.

Care is taken to avoid race conditions by ensuring independent iterations or using atomic operations where necessary. The parallelized algorithm is shown in Algorithm 4.

Algorithm 2 Parallel SVD Algorithm using QR (OpenMP)

```

1: procedure PARALLELSVD( $D$ )
2:    $[M, N] \leftarrow \text{size}(D)$ 
3:    $A \leftarrow D^\top D$  ▷ Parallel computation using OpenMP
4:   Compute eigenvalues/vectors:
5:    $[A, V] \leftarrow \text{PARALLELQRITERATION}(A)$  ▷ Eigenvalues and eigenvectors
6:   for  $i = 1$  to  $N$  do
7:      $\lambda_i \leftarrow A_{ii}$ 
8:      $\sigma_i \leftarrow \sqrt{\max(\lambda_i, 0)}$ 
9:   end for
10:   $\Sigma \leftarrow \text{diag}(\sigma_1, \dots, \sigma_N)$ 
11:   $U \leftarrow \text{zeros}(M, N)$ 
12:  for  $i = 1$  to  $N$  do
13:     $U[:, i] \leftarrow DV[:, i]/\sigma_i$ 
14:  end for
15:  return  $U, \Sigma, V^\top$ 
16: end procedure
17:
18: function PARALLELQRITERATION( $A$ )
19:   $V \leftarrow I_N$ 
20:  for  $k = 1$  to max iterations do
21:     $[Q, R] \leftarrow \text{PARALLELMGS}(A_k)$ 
22:     $A_{k+1} \leftarrow RQ$  ▷ Parallel matrix multiplication
23:     $V \leftarrow VQ$  ▷ Parallel update of eigenvectors
24:    if norm diag diff( $A_{k+1}, A_k$ ) < tolerance then
25:      break
26:    end if
27:  end for
28:  return  $A, V$ 
29: end function
30:
31: function MGS_PARALLEL( $A$ )
32:   $[m, n] \leftarrow \text{size}(A)$ 
33:   $Q \leftarrow \text{zeros}(m, n), \quad R \leftarrow \text{zeros}(n, n)$ 
34:  for  $i = 1$  to  $n$  do
35:     $v_i \leftarrow A(:, i)$ 
36:  end for
37:  for  $i = 1$  to  $n$  do
38:     $r_{ii} \leftarrow \|v_i\|_2$  ▷ parallel reduction
39:     $q_i \leftarrow v_i/r_{ii}$ 
40:    for all  $j = i + 1$  to  $n$  in parallel do
41:       $r_{ij} \leftarrow q_i^\top v_j$  ▷ Parallel dot product
42:       $v_j \leftarrow v_j - r_{ij} \cdot q_i$  ▷ Parallel vector subtraction
43:    end for
44:     $Q(:, i) \leftarrow q_i$ 
45:  end for
46:  return  $Q = [q_1, q_2, \dots, q_n], \quad R = [r_{ij}]$ 
47: end function

```

Algorithm 3 Serial Golub-Kahan SVD

```
1: procedure SVD_GOLUBKAHAN_SERIAL( $M$ )
2:    $[U, V] \leftarrow$  identity matrices matching  $M$ 
3:   Reduce  $M$  to bidiagonal form:
4:   for  $i = 0$  to  $\min(m, n) - 1$  do
5:     Apply Householder reflection from left to  $M$ , accumulate into  $U$ 
6:     Apply Householder reflection from right to  $M$ , accumulate into  $V$ 
7:   end for
8:   Initialize diagonal ( $s$ ) and superdiagonal ( $e$ ) from bidiagonal  $M$ 
9:   while not converged do
10:    Identify active submatrix where  $e_i$  are nonzero
11:    Perform implicit zero-shift QR step on active submatrix
12:  end while
13:  Sort singular values and associated vectors
14:  return  $U$ , singular values,  $V$ 
15: end procedure
```

Algorithm 4 Parallel Golub-Kahan SVD

```
1: procedure SVD_GOLUBKAHAN_PARALLEL( $M$ )
2:    $[U, V] \leftarrow$  identity matrices matching  $M$ 
3:   Parallel for each Householder vector computation:
4:   for  $i = 0$  to  $\min(m, n) - 1$  do
5:     Apply Householder reflection from left to  $M$ , parallelized across rows
6:     Apply Householder reflection from right to  $M$ , parallelized across columns
7:   end for
8:   Extract bidiagonal  $M$  into  $s$  and  $e$ 
9:   while not converged do
10:    Identify active submatrix where  $e_i$  are nonzero
11:    Perform parallel implicit QR step:
12:      Compute Givens rotations independently for each pair  $(f, g)$ 
13:      Apply rotations to  $U$  and  $V$  rows/columns in parallel
14:    end while
15:    Make singular values non-negative and sort (parallel sort)
16:  return  $U$ , singular values,  $V$ 
17: end procedure
```

Notes:

- Householder vector computation uses parallel reductions.
- Updates to M , U , and V matrices during reflections and rotations are done with careful partitioning.
- Convergence checking is parallelized with early exits to avoid unnecessary iterations.

3 Experiments

3.1 Correctness Test

To verify the correctness of our SVD implementations, we performed two main checks after computing the decomposition:

1. **Orthogonality of U and V :** We checked that $U^T U \approx I$ and $V^T V \approx I$ within a numerical tolerance of 10^{-6} , confirming that both U and V are orthogonal matrices.
2. **Reconstruction Error:** We reconstructed the original matrix as $U \Sigma V^T$ and verified that the maximum absolute difference between the reconstructed and original matrix entries is less than 10^{-6} .

Both checks passed for all tested matrices, indicating that our implementations are numerically correct.

3.2 Setup

All experiments were conducted on a server running Ubuntu 22.04.4 LTS with 128GB RAM and 32 CPU cores. We evaluated both the QR-based and Golub-Kahan (GK) SVD algorithms.

Weak Scaling: For weak scaling experiments, we generated random matrices of increasing size and measured the time taken for SVD using both QR and GK methods. The number of threads was fixed at 8. The results are shown in Figures 1 and 2.

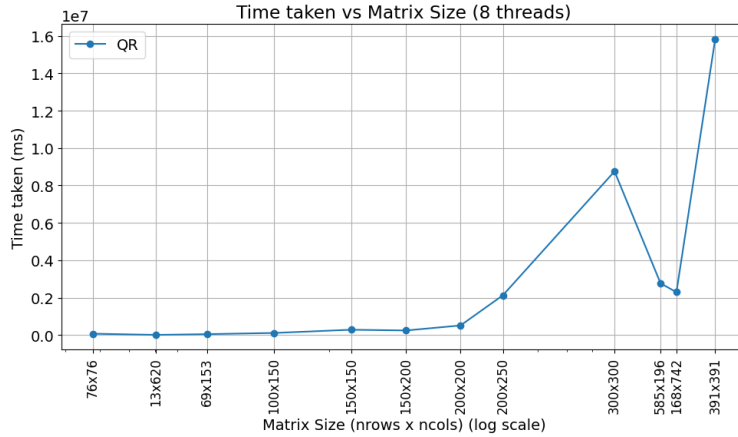


Figure 1: Weak scaling for QR-based SVD (matrix dimension and time; log scale for dimension).

Strong Scaling: For strong scaling, we fixed the matrix size to 200×200 and varied the number of threads from 1 to 64 (powers of 2). The results for QR and GK methods are shown in Figures 3 and 4.

Larger Matrix Strong Scaling: To further analyze scaling behavior, we performed strong scaling on the GK method for a larger 633×633 matrix (Figure 5). This was not done for QR due to its high computational cost for large matrices.

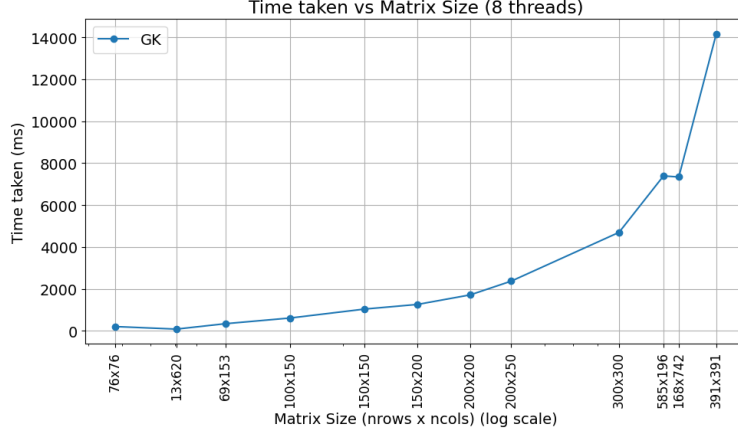


Figure 2: Weak scaling for Golub-Kahan SVD (matrix dimension and time; log scale for dimension).

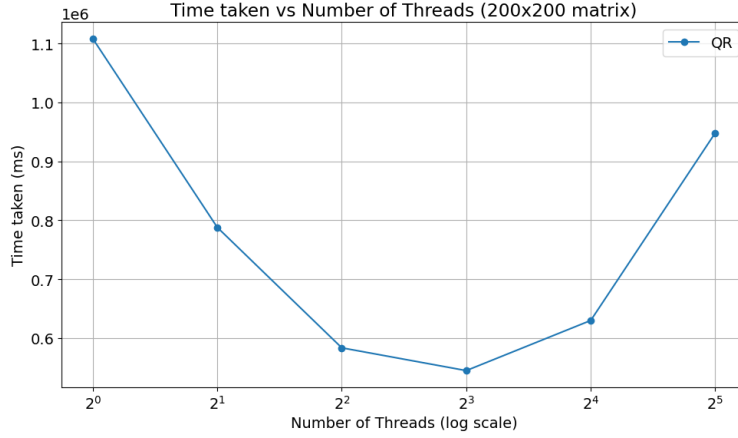


Figure 3: Strong scaling for QR-based SVD on 200×200 matrix (threads on log scale).

3.3 Results

- **Golub-Kahan (GK) vs QR:** The GK method consistently outperformed the QR method in terms of execution time. This is primarily due to the lower number of operations per iteration in the GK algorithm compared to the QR-based approach.
- **QR Anomaly for 585×196 Matrix:** The QR method took less time for a 585×196 matrix than for a 300×300 matrix. This is because the QR-based SVD operates on $A = D^\top D$, whose size depends on the number of columns of D . Thus, even though 585×196 has more rows, the effective computation is on a 196×196 matrix, which is smaller than 300×300 .
- **Sensitivity to Matrix Dimension:** The QR method is more sensitive to increases in matrix dimension than GK, again due to the higher computational complexity of QR decomposition and eigenvalue computation.
- **Strong Scaling Behavior:** For both QR and GK methods on the 200×200 matrix, the execution time decreased as the number of threads increased up to 8. Beyond 8 threads, the time started to increase due to thread management overhead dominating the computation time for this relatively small problem size.
- **Scaling on Larger Matrices:** For the 633×633 matrix (GK method), strong scaling improved with more threads, as the larger problem size allowed better utilization of parallel resources and reduced

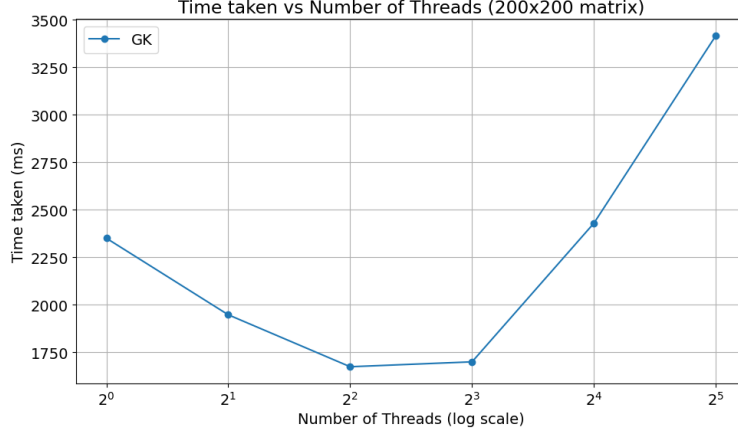


Figure 4: Strong scaling for Golub-Kahan SVD on 200×200 matrix (threads on log scale).

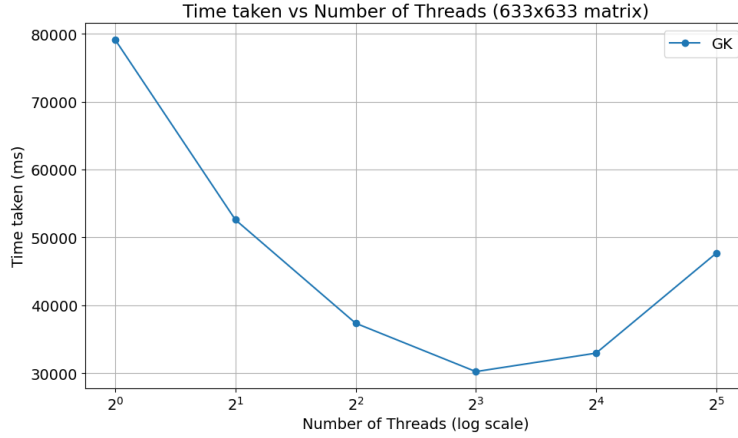


Figure 5: Strong scaling for Golub-Kahan SVD on 633×633 matrix (threads on log scale).

the relative impact of thread overhead.

4 Conclusion

In this work, we implemented and analyzed parallel algorithms for computing the Singular Value Decomposition (SVD) using both the QR-based and Golub-Kahan (GK) methods. Our experiments demonstrated that parallelization significantly reduces the computation time for SVD, especially as the matrix size increases. The GK method consistently outperformed the QR-based approach in terms of execution time, primarily due to its lower computational complexity per iteration.

We observed that the QR-based method is more sensitive to increases in matrix dimension, and its computational cost becomes prohibitive for large matrices. In contrast, the GK method scales better and benefits more from parallelization, particularly for larger matrices where thread overhead is less significant compared to the total computation time.

Our correctness tests confirmed that both implementations produce accurate results, with orthogonality and reconstruction errors well within the specified tolerance. Overall, our study highlights the effectiveness of parallelization for SVD and the practical advantages of the Golub-Kahan method for large-scale problems. Future work could explore further optimizations, distributed-memory parallelism, and applications to even larger datasets.